UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

Facultad de Ciencias | Ciencias de la Computación

Music Database

Diseño, construcción e integración de una base de datos musical

Reporte técnico del proceso de desarrollo de MusicMiner y MusicApp



Área: Diseño de software, persistencia y aplicaciones de escritorio
Plataforma: C#, .NET 8, SQLite, TagLib# y Avalonia UI
Fecha: 13 de noviembre de 2026
Alcance: Evolución de ideas, implementación, dificultades y estado actual

Resumen ejecutivo

El proyecto **Music Database** consta de dos programas en C# conectados por una base de datos SQLite. MusicMiner es un ejecutable independiente que encuentra archivos MP3 en directorios y subdirectorios, obtiene etiquetas musicales con TagLib# y registra sus metadatos. MusicApp es una aplicación gráfica de escritorio que permite explorar, buscar, agregar y editar canciones, álbumes e intérpretes, además de iniciar al minero como un proceso separado.

El diseño evoluciono desde un minero organizado en tres clases de modelo, un controlador y una vista de consola, hasta una solución con dos proyectos autónomos. Durante esa evolución se revisaron la representación de canciones como objetos, las relaciones entre tablas, el uso de una única conexión por inserción, la necesidad de transacciones, los metadatos incompletos, la separacion de errores y resultados, y la comunicación entre ejecutables mediante argumentos y JSON.

Este reporte distingue *ideas iniciales*, *decisiones efectivamente implementadas* y *limitaciones aun abiertas*. Su base documental comprende los archivos fuente compartidos de ambos proyectos, el esquema `rolas.sql`, el prototipo visual de la interfaz y el historial de decisiones de desarrollo.

Alcance de la verificacion. El análisis se realizó sobre el código fuente entregado y las pruebas relatadas durante el desarrollo. Se registro que MusicApp llego a compilar y a invocar al minero, pero no se realizó en este reporte una ejecucion automatizada completa sobre el equipo del desarrollador. Los resultados numéricos de ejemplos son ilustrativos salvo donde se indica que provienen de una prueba observada.

Índice

1. Introducción y planteamiento **1**

1.1. Problema que se pretende resolver 1

1.2. Objetivos especificos 1

1.3. Sistema de archivos, archivo SQL y base de datos 1

2. Evolución del diseño: de la propuesta al sistema integrado **1**

2.1. Primera etapa: descomponer el minero 1

2.2. Segunda etapa: la representación de Song 2

2.3. Tercera etapa: inserción relacional 2

2.4. Cuarta etapa: resultados y tolerancia a fallos 2

2.5. Quinta etapa: de Searcher anidado a un MVC de aplicación 2

3. Arquitectura general de la solución **3**

3.1. Dos programas y un almacenamiento comun 3

3.2. Separacion de responsabilidades 4

4. Funcionamiento interno de MusicMiner **4**

4.1. Entrada y validación de argumentos 4

4.2. Búsqueda recursiva y lectura de etiquetas 4
4.3. Regla de aceptacion y tratamiento de errores 5
4.4. Persistencia: GetOrCreateSong y sus dependencias 5
4.5. Limite actual del conteo de resultados 6

5. Diseño y funcionamiento de la base de datos 6
5.1. Esquema relacional recibido 6
5.2. Restricciones y convenciones del esquema 7

6. MusicApp: modelo, controladores e interfaz 7
6.1. Estructura MVC implementada 7
6.2. Búsqueda y presentacion de resultados 8
6.3. Altas, ediciones y validación 8
6.4. Evolución de la interfaz gráfica 8

7. Integración final entre MusicApp y MusicMiner 9
7.1. Contrato entre procesos 9
7.2. Rutas y despliegue 10

8. Problemas encontrados y ajustes realizados 10
8.1. Que ensenaron las pruebas realizadas 11

9. Pruebas recomendadas y estado de validación 11
9.1. Consultas de comprobacion 12

10. Limitaciones y líneas de mejora 12

11. Conclusiones 13

A. Inventario de fuentes analizadas 13

B. Arbol conceptual del repositorio 13

C. Comandos de uso y depuracion 14

D. Criterios de aceptación para la demostración 14

1. Introducción y planteamiento

1.1 Problema que se pretende resolver

Una coleccion de música distribuida entre carpetas no cuenta necesariamente con un catálogo consultable. Para encontrar una cancion, un programa tendria que recorrer archivos o mantener indices externos; la lectura repetida de etiquetas también dificulta el manejo de álbumes e intérpretes. El objetivo fue construir una solución capaz de **indexar metadatos de archivos MP3 en SQLite** y ofrecer una interfaz para consultar y administrar el catálogo resultante.

Desde el comienzo se establecio un requisito de arquitectura: el minero debía contar con **su propio ejecutable** y no formar parte del código interno del programa gráfico. Esta restriccion condiciono la organización de la solución, el intercambio de rutas y la posterior adopcion de un contrato de salida estructurada.

1.2 Objetivos especificos

1. Recorrer directorios recursivamente y filtrar archivos MP3.
2. Leer etiquetas ID3 mediante una biblioteca especializada, sin implementar desde cero su formato binario.
3. Representar los metadatos en objetos y persistirlos en un esquema relacional existente.
4. Permitir consultas, altas y ediciones desde una interfaz gráfica de escritorio.
5. Mantener el minero desacoplado de la interfaz e invocarlo mediante un proceso externo.
6. Comunicar al usuario los resultados y los errores de cada operacion.

El formato musical considerado fue MP3 con etiquetas ID3v2; se discutio la necesidad de cubrir variantes como ID3v2.2, ID3v2.3 e ID3v2.4. La compatibilidad teórica de la biblioteca no sustituye pruebas con archivos de cada variante. El proyecto no acredita por si mismo la implementación de un reproductor de audio: su funcionalidad central consiste en **catalogar y administrar metadatos**.

1.3 Sistema de archivos, archivo SQL y base de datos

Una distincion conceptual relevante fue separar: (i) el *sistema de archivos*, que contiene los MP3 originales; (ii) `rolas.sql`, que específica tablas e inserciones iniciales; y (iii) un archivo `.db`, que conserva los registros reales después de ejecutar consultas e inserciones. Ejecutar nuevamente el script sobre una base ya inicializada provoca errores de tabla existente o claves repetidas; consultar una base existente requiere abrir el archivo `.db`, no volver a importar su esquema.

2. Evolución del diseño: de la propuesta al sistema integrado

2.1 Primera etapa: descomponer el minero

La primera propuesta planteó un MVC reducido al ejecutable de minado. En *Model* se ubicarian `Search`, `Analyze` y `AddDataBase`; en *Controller*, `Miner`; y en *View*, `Message`. La clase controladora debía coordinar las tres operaciones y reportar si una falla se había originado en la búsqueda, el análisis o el almacenamiento.

La revisión del alcance revelo que el procesamiento básico podia ser muy pequeno porque .NET ofrece enumeracion recursiva de archivos y TagLib# ofrece lectura de etiquetas. Por ello se evito crear algoritmos de exploracion o analizadores binarios propios, **sin renunciar a separar responsabilidades**. La

decision final fue conservar las clases identificables, pero limitar sus métodos públicos a las operaciones necesarias.

2.2 Segunda etapa: la representación de Song

Se propusieron inicialmente numerosos campos asociados a etiquetas musicales: artista, título, álbum, artista del álbum, año, número y total de pista, disco, género, comentario, compositor, intérprete original, portada y fotografia del artista. Esta enumeracion permitió distinguir la capacidad del formato ID3 de los campos realmente exigidos por la base.

La primera version de `Song` utilizó campos públicos y métodos `Get...()` al estilo Java. En la evolución hacia C# se sustituyeron por **propiedades**, se consideraron tipos anulables para año y pista y se conservaron `string[]` para artistas, artistas del álbum y géneros, ya que una etiqueta puede devolver multiples valores. El modelo final del minero incluye `Title`, `Artist`, `Year`, `Album`, `AlbumArtist`, `Track`, `Genre` y `Path`.

Tambien surgio la idea de modelar `Album`, `Artist` y `Person` como entidades independientes. Se preciso que *Artist* y *AlbumArtist* no son siempre equivalentes, especialmente en compilaciones. Sin embargo, crear una entidad `Person` no permitiria inferir datos biográficos a partir de un MP3: las etiquetas aportan nombres de intérpretes, no necesariamente fechas, integrantes o vinculos entre bandas. En consecuencia, MusicMiner conserva un objeto `Song` para transportar etiquetas y deja la resolucion relacional a su clase de persistencia. Por su parte, MusicApp si usa modelos `Song`, `Artist`, `Album` y `PersonInfo` para operar con registros de SQLite.

2.3 Tercera etapa: inserción relacional

La primera estrategia considerada fue **analizar toda la coleccion, insertar primero todos los intérpretes y álbumes, y luego las canciones**. Posteriormente se adopto un flujo incremental: para cada MP3 valido se comprueba si la cancion ya existe, se obtiene o crea el intérprete, se obtiene o crea el álbum y se inserta la cancion con sus identificadores. Esto reduce la necesidad de almacenar previamente en memoria la coleccion completa.

En una version preliminar, `GetOrCreateSong`, `GetOrCreatePerformer` y `GetOrCreateAlbum` abrian conexiones separadas. El método se refactorizo para **abrir una sola conexión y una transacción por cancion**, compartidas con los métodos auxiliares. La transacción permite revertir cambios relacionados si falla la inserción. Esta decision no equivale a mantener una única conexión durante el minado entero: la conexión se crea y cierra para cada archivo procesado.

2.4 Cuarta etapa: resultados y tolerancia a fallos

Originalmente `Miner.Mina()` devolveria una cadena con éxito o fracaso. Se identifico que una ejecucion puede ser parcialmente satisfactoria: por ejemplo, se reporto una prueba con seis MP3, uno procesado y cinco omitidos por datos incompletos. Se introdujo `MiningResult` con contadores y motivos, y se dejo a `Message` la presentacion del resumen en consola.

Con esta separacion, el controlador coordina el proceso y la vista representa resultados, en vez de mezclar `Console.WriteLine()` con la lógica. Posteriormente `Program` incorporo el modo `--json`, necesario para que la aplicación gráfica no dependa de interpretar texto libre.

2.5 Quinta etapa: de Searcher anidado a un MVC de aplicación

Para MusicApp se propuso en principio una carpeta `View` y otra llamada `Searcher`, esta última con sus propias carpetas `Model`, `Controller` y `View`. El modelo contendria `FindSong`, `FindArtist`

y `FindAlbum`; el controlador `Finder.FindGlobal()` identificaria el objeto solicitado; y una clase `ErrorFind` devolveria mensajes si no aparecian coincidencias.

El diseño se replanteó para disponer de **un solo MVC en MusicApp**, con `Find` para lecturas, `LibraryEditor` para escrituras, `Finder` para consultas, `LibraryController` para validaciones y cambios, y `MiningController` para el proceso externo. Se reemplazo la idea de pasar un objeto incompleto a `FindGlobal()` por un texto y el enumerado `SearchType`; los resultados se devuelven como colecciones tipadas, no como un único objeto. No encontrar registros paso a considerarse una *respuesta valida vacia*, distinta de un error técnico. Se sustituyó así la necesidad inicial de `ErrorFind` por el mecanismo `Result<T>` y los mensajes propios de la vista.

Cuadro 1: *Principales decisiones y reformulaciones del desarrollo.*

Aspecto	Propuesta inicial	Diseño posterior
Análisis ID3	Implementar un analizador dentro de Model	Delegar la lectura de etiquetas a TagLib#
Objeto musical	Campos públicos, getters manuales	Propiedades, arreglos y tipos anulables
Persistencia	Varias pasadas e inserciones por etapas	Insercion incremental con `GetOrCreate...`
Conexiones	Una conexión en cada método auxiliar	Una conexión y transacción por cancion
Resultado del minero	Cadena de éxito o fallo	`MiningResult` y salidas texto/JSON
Buscador	MVC anidado y objeto como selector	MVC único, `SearchType` y resultados tipados
Errores de búsqueda	Clase `ErrorFind`	Listas vacias y `Result<T>` diferenciado
Integración	Acceder al minero desde MusicApp	Ejecutar `MinerExecutable` como proceso externo

3. Arquitectura general de la solución

3.1 Dos programas y un almacenamiento comun

La separacion entre ejecutables es una decision estructural del proyecto. MusicMiner realiza una tarea por lotes sobre archivos; MusicApp administra operaciones interactivas y la presentacion. Ambos pueden acceder a **la misma base SQLite**; no comparten clases internas ni necesitan compilarse como una sola aplicación.

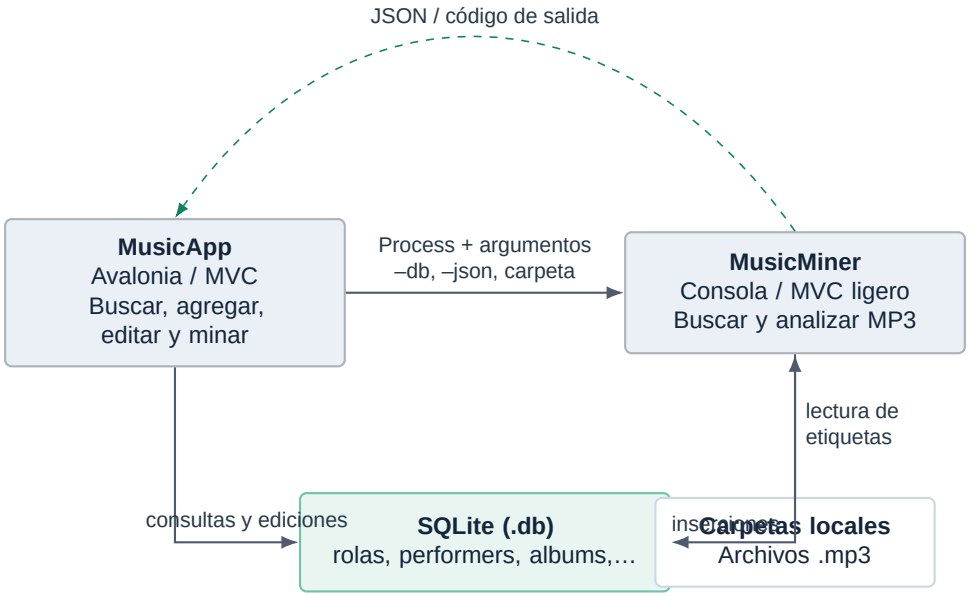


Figura 1: *Vista global: procesos independientes, datos compartidos y comunicación mediante JSON.*

3.2 Separacion de responsabilidades

En el minero, `Search` encuentra rutas, `Analyze` convierte metadatos en un objeto `Song`, `AddDataBase` persiste y `Miner` orquesta. En la aplicación gráfica, `Find` y `LibraryEditor` ejecutan SQL, los controladores coordinan las solicitudes y las vistas Avalonia muestran resultados. Es una aplicación de MVC en sentido practico, no un requisito de que cada comportamiento tenga una clase independiente.

Dos modelos Song, dos necesidades. MusicMiner emplea un `Song` orientado a etiquetas, que conserva `string[]` para intérpretes y géneros. MusicApp usa un `Song` de consulta con identificadores SQLite, nombres resueltos y campos escalares. Esta diferencia refleja la separacion entre *extracción de archivos* y *administracion de registros*.

4. Funcionamiento interno de MusicMiner

4.1 Entrada y validación de argumentos

El punto de entrada es `Program.cs`, perteneciente al proyecto `MinerExecutable.csproj` con destino `net8.0`. Interpreta `--db` para la base, `--json` para la salida estructurada y un argumento posicional para la carpeta musical. Convierte ambas rutas a rutas absolutas y valida que la base y el directorio existan antes de iniciar `Miner.Mina()`.

```
dotnet run -- --db "/ruta/rolas.db" --json "/ruta/Musica"
```

Sin `--json` se instancia `Message` y se imprime un resumen legible. En modo JSON, `Program` serializa los contadores y motivos de `MiningResult`, permitiendo que un programa externo procese la salida.

4.2 Búsqueda recursiva y lectura de etiquetas

`Search.FindMp3Files()` utiliza `Directory.EnumerateFiles` con `RecurseSubdirectories=true`, `IgnoreInaccessible=true` y coincidencia de mayúsculas y minúsculas indiferente. Devuelve un

`IEnumerable<string>`, con lo que el controlador puede procesar las rutas a medida que las recorre, sin construir por necesidad una lista global de canciones.

`Analyze.ReadMeData()` abre cada archivo mediante `TagLib.File.Create(path)` y lo libera con `using`. Obtiene título, `Performers`, año, álbum, `AlbumArtists`, `Genres`, número de pista y ruta. Su única salida es un objeto `Song`; no abre conexiones SQL.

4.3 Regla de aceptacion y tratamiento de errores

El controlador `Miner.Mina()` exige título, álbum y primer intérprete no vacios. Si falta uno de ellos, incrementa `Skipped`, agrega el motivo a `Errors` y continua con el siguiente archivo. Si ocurre una excepción durante lectura o almacenamiento, también registra el mensaje y continua; una excepción general del recorrido se agrega al resultado.

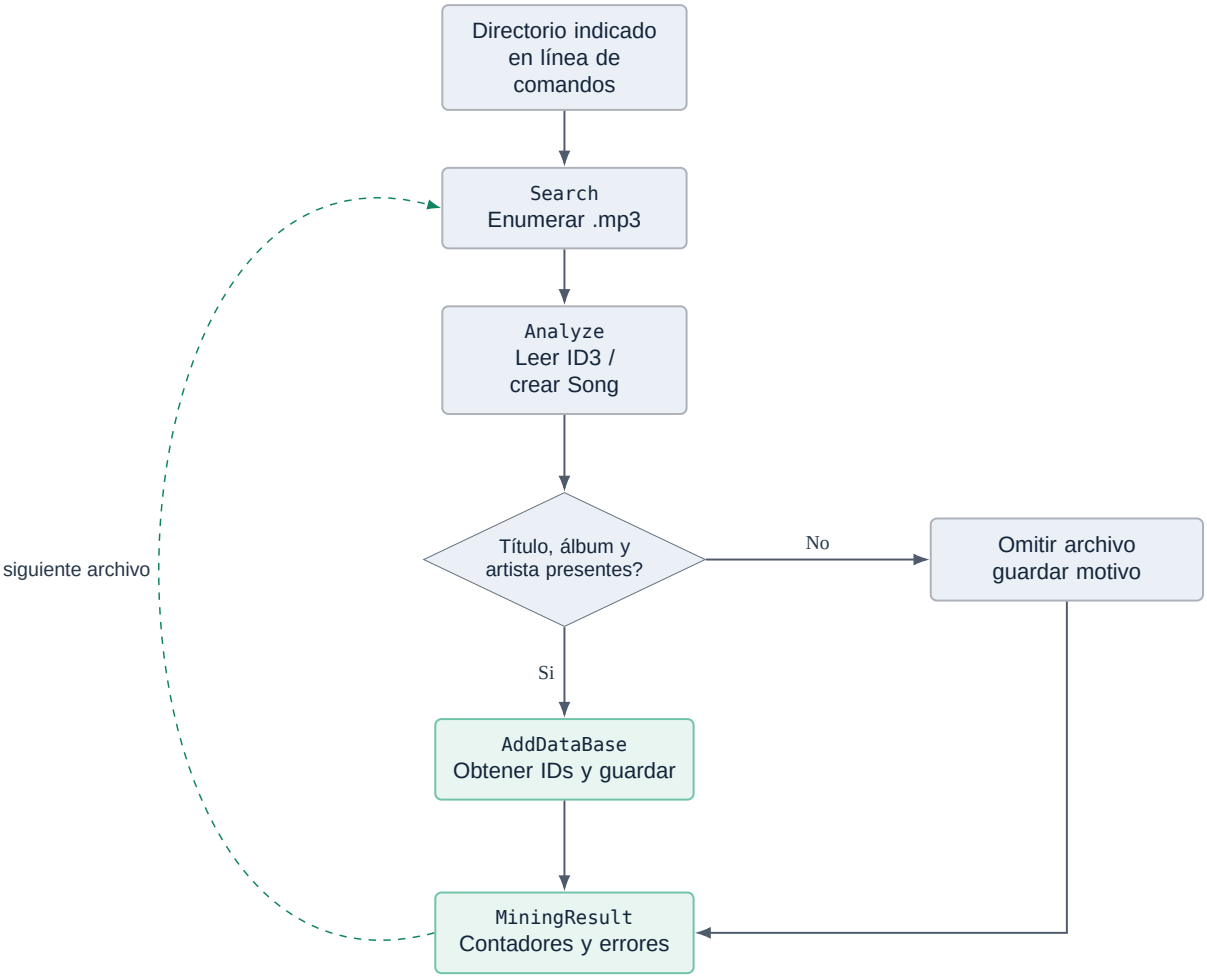


Figura 2: *Flujo de control por archivo: validación, persistencia y omisión recuperable.*

4.4 Persistencia: GetOrCreateSong y sus dependencias

`GetOrCreateSong()` abre una conexión `SqliteConnection` y una transacción. Primero ejecuta `SELECT id_rola FROM rolas` con la condición `path = $path`. Si la ruta ya existe, devuelve ese identificador. Si no existe, busca o crea el intérprete en `performers` y el álbum en `albums`; usa sus IDs para insertar en `rolas`. Al finalizar confirma la transacción.

`GetOrCreatePerformer()` consulta por nombre y, si es necesario, crea un intérprete con `id_type=2`

(desconocido). `GetOrCreateAlbum()` utiliza nombre y ruta de carpeta para buscar o crear el álbum. El año se copia de la etiqueta de la cancion; debe reconocerse que no necesariamente es el año de lanzamiento original del álbum.

En el objeto `Song` pueden existir varios intérpretes y géneros, pero la inserción actual conserva solo el primer elemento de cada arreglo. Los datos biográficos de `persons`, `groups` e `in_group` no se derivan ni completan en esta etapa.

Granularidad de la transacción. La integridad se protege por cancion procesada, no por coleccion completa. Si falla un `INSERT`, las modificaciones pendientes de esa cancion pueden revertirse; las canciones ya confirmadas permanecen en SQLite.

4.5 Limite actual del conteo de resultados

En la implementación examinada, `GetOrCreateSong()` devuelve un ID tanto cuando inserta una fila como cuando encuentra una existente. `Miner.Mina()` incrementa `Registered` en ambos casos, de modo que esa propiedad cuenta operaciones satisfactorias, no necesariamente *inserciones nuevas*. A su vez, `Program.cs` serializa `duplicates=0` y `failed=0` como constantes. El programa también puede devolver código de salida 0 después de que un error general haya sido capturado internamente por `Miner`. Estos comportamientos forman parte de las limitaciones documentadas y deben diferenciarse del contrato ideal de la interfaz.

5. Diseño y funcionamiento de la base de datos

5.1 Esquema relacional recibido

El esquema de trabajo contiene siete tablas. Las principales son `rolas`, `performers`, `albums` y `types`. Ademas se consideran `persons`, `groups` e `in_group` para información de personas, bandas y sus integrantes. `rolas` guarda las claves `id_performer` e `id_album` en vez de repetir directamente todos los atributos de estas entidades.

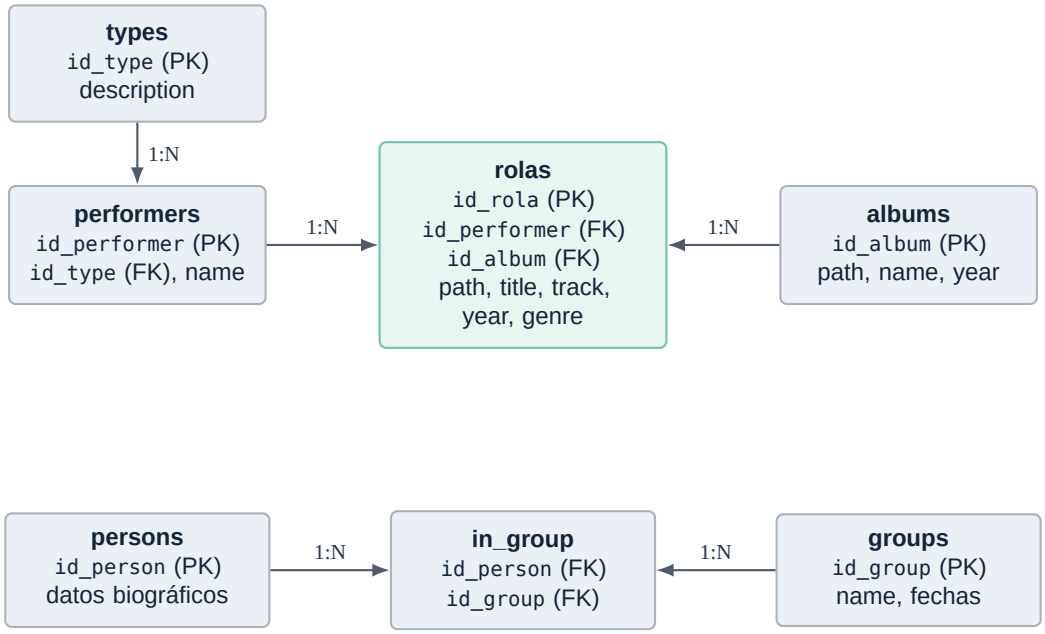


Figura 3: *Relaciones declaradas por el esquema. No existe una clave foranea explicita entre `persons` y `performers`.*

Cuadro 2: *Papel de las principales tablas del esquema.*

Tabla	Informacion almacenada
`rolas`	Cancion, ruta, pista, año, género y referencias al intérprete y álbum.
`performers`	Nombre del intérprete y tipo musical asociado a `types`.
`albums`	Identificador, ruta, nombre y año del álbum.
`types`	Catalogo con Person (0), Group (1) y Unknown (2).
`persons`	Nombre de escenario, nombre real y fechas biográficas.
`groups`	Nombre del grupo y fechas de actividad.
`in_group`	Relacion entre una persona y un grupo.

5.2 Restricciones y convenciones del esquema

El script `rolas.sql` entregado contenia una omisión sintáctica en la declaracion de `in_group`: faltaba una coma entre dos clausulas `FOREIGN KEY`. Su corrección era necesaria para crear el esquema completo desde cero. Ademas, el script incluye instrucciones `INSERT` iniciales; ejecutarlo repetidamente sobre una base ya creada produce conflictos de tabla existente y claves `PRIMARY KEY` repetidas.

Otro limite del esquema es la ausencia de un vinculo relacional explicito entre `persons` y `performers`. En MusicApp se adopto la convención `persons.id_person = performers.id_performer` para el caso Person. Es una *convención de aplicación*, no una relación que la base garantice por si misma; debe documentarse y revisarse antes de usar datos externos.

En el minero, la identidad de una cancion se basa en `path`; el esquema recibido no declara una restriccion `UNIQUE` sobre dicha columna. Esta distincion explica por que la consulta previa reduce duplicados secuenciales, pero no garantiza unicidad ante rutas diferentes o accesos concurrentes. Si se incorpora un índice único, primero hay que revisar y depurar duplicados ya existentes.

6. MusicApp: modelo, controladores e interfaz

6.1 Estructura MVC implementada

El paquete de MusicApp contiene un proyecto Avalonia 11.2.3 sobre .NET 8. En lugar de colocar otro MVC dentro de `Searcher`, se desarrollo una sola jerarquia de `Model`, `Controller` y `View`. `AppConfig` determina las rutas de SQLite y del ejecutable; prioriza argumentos `--db` y `--miner`, después variables `MUSICAPP_DB` y `MUSICMINER_PATH`, y por último valores predeterminados.

Cuadro 3: *Responsabilidades efectivamente visibles en MusicApp.*

Capa	Componentes	Funcion
Model	`Song`, `Artist`, `Album`, `PersonInfo`	Objetos consultables o editables con IDs de SQLite.
Model	`Find`, `LibraryEditor`, `Database`	Consultas, cambios transaccionales y conexión controlada.
Controller	`Finder`, `LibraryController`	Seleccion de búsqueda, validación y administracion.
Controller	`MiningController`, `DbAccess`, `Result<T>`	Proceso externo, manejo de conexiones y errores tipados.
View	`MainWindow`, `SearchView`, `LibraryView`, formularios	Navegacion, listas, captura de datos y mensajes.

6.2 Búsqueda y presentacion de resultados

`Finder.FindGlobal(searchText, searchType)` recibe texto y `SearchType` (Song, Artist o Álbum). Segun el tipo invoca los métodos de `Find`, cuyos resultados se empaquetan en `SearchResults`: listas de canciones, intérpretes o álbumes. La capa de lectura usa `LIKE` parametrizado, escape de caracteres comodin y, para canciones, `LEFT JOIN` con `performers` y `albums`. Asi se obtienen datos suficientes para representar título, intérprete y álbum sin que la vista genere SQL.

`DbAccess.Run` concentra la apertura y cierre de conexiones y traduce excepciones a `Result<T>`. Un resultado vacio es una consulta exitosa sin coincidencias, mientras que los errores de validación, disponibilidad de base, duplicados o fallos de SQL se expresan como estados distintos. Las vistas convierten modelos a filas mediante `ResultRow` y muestran mensajes de acuerdo con ese estado.

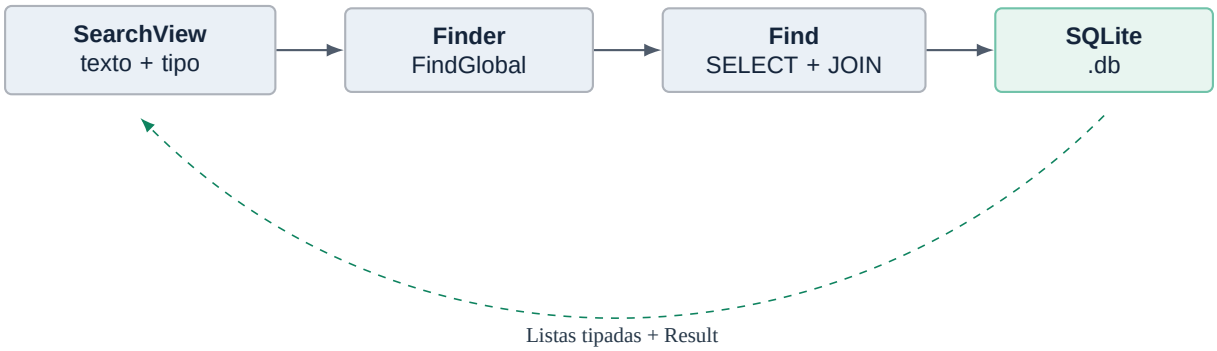


Figura 4: *Recorrido conceptual de una búsqueda; la vista nunca ejecuta SQL directamente.*

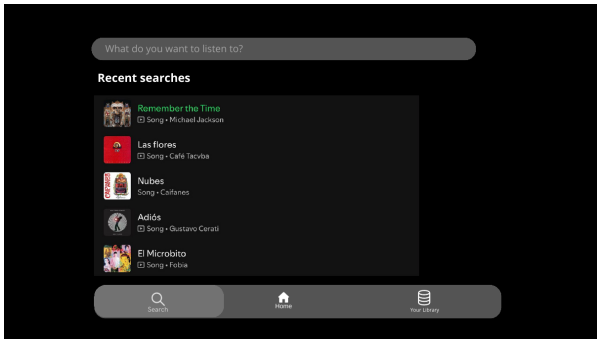
6.3 Altas, ediciones y validación

`LibraryController` recibe textos crudos de formularios, valida campos obligatorios y convierte pista y año a enteros opcionales. `LibraryEditor` implementa operaciones de inserción y actualizacion para las tres entidades principales. Cada operacion de escritura utiliza una transacción y consultas parametrizadas; las canciones se editan por `id_rola`, no solo por título. `ResolvePerformer` busca por nombre, `ResolveAlbum` reutiliza un álbum existente o el álbum reservado 0 cuando no se indica nombre.

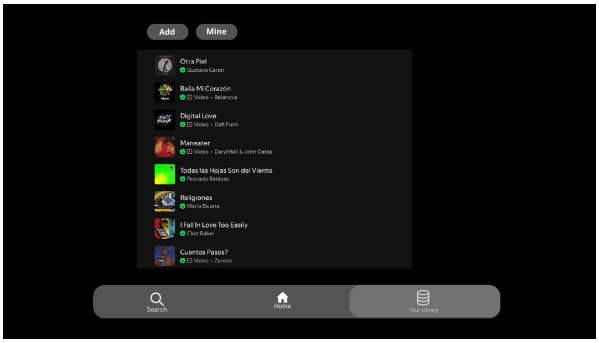
En el caso de intérpretes tipo Person, la vista despliega un segundo paso para los datos personales. Esos datos se guardan en `persons` mediante la convención de identificadores ya descrita. Los grupos se representan en `performers`, pero el prototipo implementado no administra automáticamente las relaciones de integrantes de `in_group`.

6.4 Evolución de la interfaz gráfica

El prototipo visual planteó una pantalla inicial con acciones de edición y cierre; una búsqueda con historial; una biblioteca con botones Add y Mine; formularios para Artist, Song y Álbum; un formulario más detallado para Person; y un flujo de búsqueda previa a la edición. La estetica propuesta fue negra, gris y verde, con tarjetas de esquinas redondeadas y una barra inferior de navegación Search–Home–Your Library.



Búsqueda e historial: propuesta visual



Biblioteca con acciones Add y Mine

Figura 5: *Dos bocetos originales de interfaz incorporados como referencia de diseño.*

Al implementar las vistas se mantuvieron las secciones centrales, la navegación y los formularios. Se replantearon algunas funcionalidades que carecian de soporte en la base: el historial persistente de búsquedas, las carátulas y las fotografias de intérpretes no forman parte del código confirmado; los campos visuales *Add picture* no se conectaron a almacenamiento. Las operaciones *Add* y *Edit*, en cambio, quedaron representadas con controladores especificos, validaciones e IDs reales de SQLite.

Tecnologia de interfaz. Avalonia permite conservar C# en Linux y separar XAML (composición visual) de C# (eventos de la vista). En esta aplicación se empleo una organización MVC explicita, aunque Avalonia también facilita patrones MVVM.

7. Integración final entre MusicApp y MusicMiner

7.1 Contrato entre procesos

La integración no consiste en copiar las clases de MusicMiner al proyecto gráfico. `LibraryView` utiliza un selector nativo de directorios y pasa la ruta a `MiningController.MineAsync()`. Este controlador verifica directorio, base de datos y ejecutable; construye los argumentos mediante `ArgumentList` de `ProcessStartInfo`; ejecuta `MinerExecutable`; captura `stdout` y `stderr` en paralelo; espera su finalización sin bloquear directamente la interfaz; y deserializa la última línea JSON que cumpla el formato esperado.

```
MinerExecutable --db "/ruta/rolas.db" --json "/ruta/Musica"
```

La respuesta JSON contiene `found`, `inserted`, `duplicates`, `skipped`, `failed` y `reasons`. El controlador de MusicApp exige un código de salida satisfactorio y un JSON interpretable; en caso contrario informa un error. Si el proceso finaliza satisfactoriamente, la vista recarga las canciones desde SQLite y presenta `MiningResultView`.

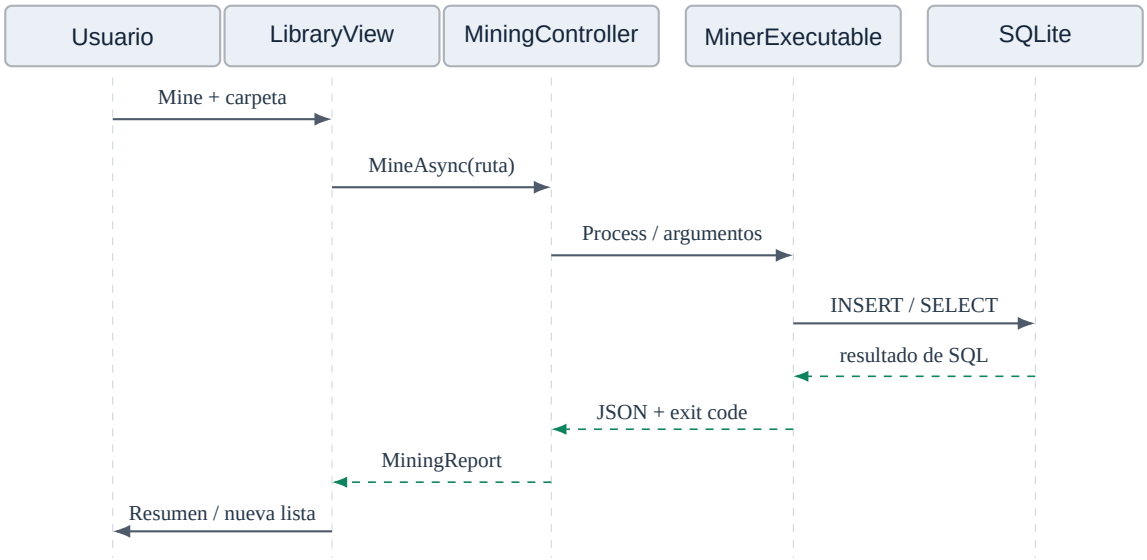


Figura 6: *Secuencia de ejecucion del minado solicitado desde la interfaz.*

7.2 Rutas y despliegue

Durante el desarrollo aparecio una confusa diferencia entre *carpeta del proyecto* y *nombre del ejecutable*. Aunque el código vive en un directorio `MusicMiner`, el archivo compilado es `MinerExecutable`; la configuración de MusicApp debe apuntar a esa ruta real, por ejemplo el ejecutable ubicado bajo `bin/Debug/net8.0/`. Tambien se distinguio la ruta del directorio que se analiza de la ruta del archivo `.db`: ambos programas deben apuntar a la misma base de datos, pues una base bajo `~/.local/share/MusicApp/` es diferente de otra creada en el directorio de desarrollo.

La ejecucion final de la integración fue confirmada en una prueba manual: MusicApp consiguió iniciar el minero. Sin embargo, la observacion posterior de canciones aparentemente repetidas motivo una nueva revisión del criterio de identidad y del origen de los datos que se mostraban.

8. Problemas encontrados y ajustes realizados

Cuadro 4: *Incidencias documentadas, su causa y las medidas de corrección o diagnostico.*

Incidencia	Causa identificada	Respuesta adoptada o pendiente
Lectura de MP3 sin etiquetas	Archivos descargados desde convertidores no incluian título, álbum u otros metadatos. La prueba con `demo.mp3` del curso si devolvio etiquetas.	Se incorporo validación de título, álbum e intérprete; los archivos incompletos se omiten con motivo.
`Value must be set`	Un parametro de la consulta de álbum recibia un nombre nulo en `GetOrCreateAlbum`.	Se detecto mediante el *stack trace*; la validación previa del minero evita enviar esos archivos a la inserción. La política de etiquetas faltantes puede ampliarse en el futuro.

Incidencia	Causa identificada	Respuesta adoptada o pendiente
Consultas SQL incorrectas	Primeras pruebas contenian nombres de parametros repetidos, marcador de género mal escrito y nombre del álbum usado donde correspondia su ID.	Se separaron las resoluciones de intérprete y álbum, se parametrizaron consultas y se gestionaron valores `NULL`.
Tres conexiones por inserción	Los métodos auxiliares abrian conexiones independientes.	Se refactorizo la inserción a una sola conexión y transacción compartidas por cancion.
Reimportacion de `rolas.sql`	Las tablas e IDs iniciales ya existian en `rolas.db`.	Se distinguio creacion del esquema de consulta: usar `sqlite3 archivo.db` y `SELECT`, no importar el script en cada prueba.
`AVLN2000`	Tres vistas XAML usaban `ColumnSpacing` de una forma rechazada por la compilación disponible.	Se sustituyó esa separacion visual por margenes entre controles; el proyecto volvió a compilar en el entorno de desarrollo.
`CS0050/51/53`	Los tipos `Song` expuestos por APIs publicas no tenian accesibilidad equivalente; además, se había colocado el minero bajo el proyecto MusicApp.	Se revisó la visibilidad `public` de modelos y se mantuvieron ambos proyectos en carpetas independientes, evitando inclusion accidental de fuentes.
Ejecutable no localizado	Se asumio que el ejecutable se llamaba `MusicMiner`.	Se identifico su nombre efectivo `MinerExecutable` y se paso su ruta mediante `--miner` o `MUSICMINER_PATH`.
Canciones repetidas	Se observaron canciones aparentemente duplicadas en la integración; no se confirmó si había igual ruta, rutas distintas o filas duplicadas solo en la vista.	Se propuso verificar `GROUP BY path HAVING COUNT(*)>1`, revisar rutas y considerar `UNIQUE(path)` tras depurar la base. **No se da por solucionado.**

8.1 Que ensenaron las pruebas realizadas

El caso de los archivos provenientes de convertidores evidencio que *tener audio MP3 valido no implica poseer etiquetas ID3 completas*. Por eso las pruebas deben incluir entradas validas, incompletas y corruptas, y no solo el ejemplo `demo.mp3`. Asimismo, los errores de compilación mostraron la importancia de compilar cada ejecutable por separado y conservar la division de proyectos. Finalmente, la aparicion de duplicados demostro que el éxito del proceso y la corrección de los registros no son equivalentes: deben verificarse consultas directas a SQLite.

9. Pruebas recomendadas y estado de validación

El diseño admite pruebas unitarias de las clases y pruebas de integración sobre una base temporal. La siguiente matriz separa lo *observado durante el desarrollo* de lo que *aun debe verificarse de forma sistematica*.

Cuadro 5: *Matriz de verificacion y evidencia disponible.*

Prueba	Estado	Evidencia o criterio de aceptacion
Compilar MusicApp	Reportada como exitosa	Los errores XAML previos dejaron de presentarse.
Invocar MusicMiner desde MusicApp	Reportada como exitosa	El minero se ejecuta como proceso externo.
Leer `demo.mp3`	Observada	Etiquetas de ejemplo (título T, álbum L, año 1950, género Jazz) fueron leidas.
MP3 sin metadatos	Observada	Cinco archivos fueron omitidos tras el diagnostico; revisar motivos.
ID3v2.2, v2.3 y v2.4	Pendiente de bateria formal	Usar archivos controlados de cada variante y verificar etiquetas.
Insertar dos veces la misma ruta	Por verificar	La cantidad de filas para esa ruta debe permanecer en 1.
Controlar copias en rutas distintas	Pendiente de definir	Acordar si son dos archivos validos o duplicados de contenido.
Fallos de inserción y transacciones	Pendiente	Ninguna inserción parcial de la misma cancion debe permanecer.
Alta y edición desde formularios	Pendiente de bateria formal	Los cambios deben reflejarse en `SELECT` por ID.
Consistencia de estadisticas JSON	Pendiente de ajuste	`inserted`, `duplicates` y `failed` deben ser contadores reales.

9.1 Consultas de comprobacion

Las siguientes consultas pueden ejecutarse sobre **una copia de la base de datos** de pruebas. Sirven para revisar resultados sin depender exclusivamente de lo que muestre la interfaz.

```
SELECT id_rola, title, path FROM rolas ORDER BY id_rola DESC;
SELECT path, COUNT(*) AS repeticiones
FROM rolas GROUP BY path HAVING COUNT(*) > 1;
PRAGMA foreign_key_check;
```

10. Limitaciones y líneas de mejora

1. **Definir identidad de canciones.** La comprobacion por ruta exacta no detecta copias identicas en otras carpetas y no basta contra inserciones concurrentes; evaluar normalizacion de rutas, restricciones unicas o hashes, conforme a los requisitos.
2. **Corregir estadisticas.** Devolver no solo el ID de `GetOrCreateSong`, sino también si se inserto o se reutilizo. Separar `Skipped` por metadatos insuficientes de `Failed` por excepción.
3. **Corregir el estado de salida.** Un fallo general atrapado dentro de `Miner.Mina` debe reflejarse en el estado devuelto al proceso solicitante.
4. **Alinear criterios de álbum.** El minero usa nombre y ruta de carpeta, mientras que la inserción manual de MusicApp busca por nombre; definir una identidad consistente para evitar álbumes ambiguos.
5. **Reforzar las relaciones de personas y grupos.** El esquema recibido no conecta explicitamente personas con intérpretes; ampliar la base o formalizar la convención para impedir colisiones.
6. **Agregar campos gráficos futuros.** Para carátulas, fotos de artistas e historial de búsquedas sería necesario decidir almacenamiento y reglas de actualizacion.

7. **Completar la cobertura de etiquetas.** Validar ID3v2.2/2.3/2.4, varios artistas, varios géneros, años ausentes, archivos corruptos y caracteres Unicode.
8. **Verificar el despliegue.** Resolver automáticamente la ruta de `MinerExecutable` en compilaciones distribuidas y documentar las opciones `--miner` y `--db`.

11. Conclusiones

El proyecto fue evolucionando desde una tarea aparentemente simple —encontrar MP3 y guardarlos en una tabla— hasta una arquitectura de dos programas que comparte una base relacional. La dificultad principal no estuvo en la enumeracion de archivos, que ya resuelve .NET, sino en definir **limites de responsabilidad**: que hace cada clase, como se transportan los metadatos, como se resuelven las claves foraneas, como se distinguen errores y omisiones y como se comunica un ejecutable con otro.

Las decisiones de diseño evolucionaron conforme aparecieron casos reales. Se mantuvo la division entre `Search`, `Analyze` y `AddDataBase`; se introdujo un objeto `Song` específico para etiquetas; se sustituyeron conexiones repetidas por una transacción compartida por cancion; se paso de un mensaje único a resultados estructurados; y se reorganizo MusicApp para evitar un MVC anidado. La integración mediante argumentos, JSON y `ProcessStartInfo` permitió preservar la independencia del minero.

El estado documentado confirma una **integración funcional básica** y una interfaz capaz de operar con SQLite, pero también muestra asuntos por concluir: identidad de canciones, contadores confiables, manejo explicito de fallos generales y cobertura sistematica de versiones ID3. Esta distincion entre lo implementado y lo pendiente es parte de una evaluación técnica rigurosa del proyecto.

A. Inventario de fuentes analizadas

Componente	Archivos utilizados para analizar su funcionamiento
MusicMiner	`Program.cs`, `Miner.cs`, `Search.cs`, `Analyze.cs`, `Song.cs`, `AddDataBase.cs`, `MiningResult.cs`, `Message.cs`, `MinerExecutable.csproj`, `README_MusicMiner.md`.
MusicApp	`AppConfig.cs`, `Program.cs`, `MusicApp.csproj`, `Model/Find.cs`, `Model/LibraryEditor.cs`, `Model/Database.cs`, `Controller/Finder.cs`, `Controller/LibraryController.cs`, `Controller/MiningController.cs`, `Controller/Result.cs`, `Controller/DbAccess.cs`, y vistas Avalonia.
Esquema y bocetos	`rolas.sql` e `InterfazApp.pdf`.

B. Arbol conceptual del repositorio

```
proyecto2/
|-- MusicApp/
| |-- Model/ Song, Artist, Album, Find, LibraryEditor...
| |-- Controller/ Finder, LibraryController, MiningController...
| |-- View/ XAML, formularios, biblioteca y resultados
| `-- MusicApp.csproj
|-- MusicMiner/
| |-- model/ Song, Search, Analyze, AddDataBase, MiningResult
| |-- controller/ Miner
| |-- view/ Message
| `-- MinerExecutable.csproj
```

```
`-- rolas.sql Script de esquema; no es el archivo .db
```

C. Comandos de uso y depuracion

```
# Compilar proyectos de manera independiente
cd MusicMiner && dotnet build
cd ../MusicApp && dotnet build

# Ejecutar minero desde su directorio
./bin/Debug/net8.0/MinerExecutable \
  --db "/ruta/a/rolas.db" --json "/ruta/a/mp3"

# Iniciar MusicApp indicando la ubicacion real del minero
dotnet run -- \
  --db "/ruta/a/rolas.db" \
  --miner "/ruta/a/MinerExecutable"

# Consultar la base real (sin reimportar rolas.sql)
sqlite3 "/ruta/a/rolas.db"
```

D. Criterios de aceptación para la demostración

Para presentar el proyecto, se propone realizar la siguiente secuencia sobre una copia del archivo `.db`, conservando evidencia de la terminal y de los registros resultantes:

1. Iniciar MusicApp con las opciones `--db` y `--miner` apuntando a rutas existentes.
2. Buscar por nombre una canción, un álbum y un intérprete; verificar una búsqueda sin coincidencias.
3. Agregar y editar un registro desde los formularios y comprobar su ID mediante una consulta `SELECT`.
4. Seleccionar una carpeta con MP3 etiquetados y ejecutar `Mine`; verificar que la interfaz no se bloquee y que presente el resultado.
5. Repetir el minado sobre los mismos archivos; revisar tanto los contadores como las filas reales en SQLite para detectar posibles duplicaciones.
6. Probar un archivo sin etiquetas completas y otro corrupto; comprobar que ambos produzcan un motivo y no detengan innecesariamente el resto del lote.

> Los resultados de esta demostración deben interpretarse junto con las limitaciones del código actual. En particular, un valor alto de `inserted` no garantiza que se hayan creado tantas filas nuevas y `duplicates=0` no es todavía una medida real.

Fin del reporte técnico